Technische Hochschule Ingolstadt

Bachelor Thesis

Faculty of computer science

Development of a Frontend Application for Real-Time and 24-Hour Monitoring and Recording of Electrocardiographic Data with Communication via Bluetooth Low Energy

First- and Lastname : **Eric Horacsek**

Registered on : 25.11.2024

Submitted on : xx.yy.zzzz

First examiner : Prof. Dr.-Ing. Georg Passig

Second examiner : Prof. Dr. Ulrich Margull

Affidavit

I hereby declare that I have written this thesis independently, have not used any sources or aids other than those stated, have not submitted it elsewhere for examination purposes and have expressly identified all quotations from the sources used, either verbatim or in spirit. I have marked verbatim and indirect quotations as such.

Ingolstadt, _____

Firstname Lastname

Acknowledgements

This is the sentimental part where you get to thank all the persons who were a part of your thesis journey in one or the other way!

Eric Horacsek
Ingolstadt, Deutschland
November xx 2024

Abstract

The human heart is the central organ of the circulatory system and plays a vital role in supplying the body with oxygen and nutrients. Rhythmic contractions pump blood through the body, laying the foundation for all vital functions. An essential part of medical diagnostics combined with heart function and health is the electrocardiogram (ECG). The ECG measures the electrical activity of the heart and provides valuable information about the heart rhythm aswell as possible irregularities or diseases. As technology has advanced, the way in which ECG data can be collected and monitored has also evolved significantly. Mobile health monitoring in particular now makes it possible to record ECG data not only at certain points of time, but also in real time and over longer periods of time. Wireless transmission via Bluetooth Low Energy (BLE) enables reliable and energy-efficient transfer of data to mobile devices. This opens up new possibilities for applications offering real-time and long term monitoring enabling users access to relevant information and graphical processing and alarm functions. This thesis explains the conception and implementation of an Android frontend application, which realises continuous and mobile ECG monitoring. The main programming language used for this project, is kotlin with Jetpack Compose as the Framework. Moreover this dissertation is particulary focused on implemententing the communication and data transmission via BLE aswell as realising an 24-hour-mode, which might be used for 24-hour data recognition. It also provides an broad insight into the given requirements, designprincipals, and technical challenges which accured during the implementation. Furthermore this theseis offers a glimplse into the potential for modern healthsuveillance.

Confidentialityclause

Optional.

Ingolstadt, _____
(Date)

(Signature)
Firstname Lastname

Glossary

Eric Horacsek Absolute operation.

Acronyms

D Dimensions.

RGB Red Green Blue channels.

List of figures

List of tables

List of contents

1 Introduction

This chapter will give a small introduction on what the thesis and the implemented application is about. This covers an explanation for the application's motivation and the environment the application will be used in.

1.1 Motivation

The aim of this work is to create a new version of an Android App once developed by fellow students from the faculty of Electrical and Information Technology. It's core functionallity is to receive and display ECG data in real time, via BLE. With the completion of this work, the new version will be able to receive and display ECG data in real time, via BLE. In addition it will be capable of an 24-hour mode which records ECG data for up to 24 hours, so the collected data can be analyzed later.

1.2 Scope and Framework

This paper only deals with the app development process of the new version. Therefore, any implementation of the previous work done by the students is not going to be discussed in detail. The hardware implementation is also not an element of this work . Generally speaking, this paper covers all topics that are essential for the development of the new app version. Specifically, technology stack of choice, IDE choice desicion, communication with hardware via Bluetooth low energy, structure of the user interface, real-time data monitoring, implementing the 24 hour mode. In addition, the outlook explains procedures for the implementation of future features that could not be realized within the limits of this paper.

1.3 Procedure

The second chapter (Technical Fundamentals), covers the choosen technolgies like Programming language and IDE of choice and the used versions. Aswell as an short introduction into the Android Architecture, specifically the Java Framework API. Chapter 2 provides a valuable insight into the core functionallity of the operating system, making it easier to understand the implementation. Chapter three discusses the Functional Requirements , by breaking it down into multiple sub requirements. These requirements will be taken up again in chapter four, where they are translated into code.

2 Technical Fundamentals

"The nature of the application and the required functionality of a software system should influence the choice of methods and tools used during development". Clarke [1995] This ensures that the development process is both efficient and tailored to meet the unique demands of the system. Moreover, aligning tools and methods with the application's requirements can help streamline workflows and reduce potential complexities during implementation. Consequently choosing the right programming language, aswell as IDE is essential to enable efficient development.

The following chapter Technical Fundamentals contains all technical characteristics and requirements, and how they have been fulfilled.

2.1 Selected Technologies

2.1.1 Programming Language and Framework

When searching for a suitable programming language to program Android apps, names like Java/Kotlin and Dart often appear. Both languages offer great frameworks, such as Flutter for Dart or Jetpack Compose for Kotlin. One aswell as the other are highly effective because they eliminate the limitations of XML-based UI development, enabling a more streamlined, declarative approach that combines UI and logic in a single language. Thus reduces boilerplate code, and enhances performance aswell as developer productivity. The main differnce between both frameworks lies in ressource manegement. For example Flutter is superior in cpu usage, but Jetpack Compose consumes less memory.Kusuma et al. [2023] "But since both have their respective advantages and disadvantages, it is impossible to say that one is superior to the other. The choice between Jetpack Compose and Flutter depends on the needs and preferences of the developer. If developers are more familiar with the Kotlin language and want to use new technologies, then Jetpack Compose might be a better choice." Kusuma et al. [2023] With prior practical experience in Java/Kotlin, Jetpack Compose was chosen as the framework for this work.

2.1.2 Development Enviroment

Integrated development Enviroments play a fundamental role in todays application development, providing developer with tools mandatory to debug, refactor and write code. Making programming more efficient and delightful for programmers. When developing Android Apps with state of the art frameworks like Jetpack Compose, Android Studio stands out as the most suitable option as both Jetpack compose and Android Studio are

created by Google. Android Studio is the official IDE for Android Development. With IntelliJ as base it inherits powerful code editor and developer tools, combined with Android specific features. Such as Live Edit (updates running app when code is changed) aswell as Interactive Preview which allows to test functionallity between so called Compose components (Compose will be explained in the later course of this work)[1], et cetera.

The version of Android Studio used for this thesis is **Ladybug | 2024.2.1**. Which is the latest version as this was written (13.12.2024).[2] Downloading the right version can be crucial for problem-free debuging. As some versions lead to bugs, causing the screen to freeze.[3]

2.1.3 Versions

This section presents fundamental versions of the required development resources used for this work.

The tests for this application during the development process, were done on a phone using the **Android version 13** which implies the usage of **API level 33**. Utilizing lower Android version than 12 (API Level 31,32) is not possible due to new runtime permissions added in Android 13, making it impossible to start the app[4]. Higher Android versions were not reviewed. More versions of the used ressources, are presented in table 1.

Table 1: Versions of used resources

Ressource	Version used
Android version	13
API Level	33
Koltin compiler extension version	5.15.1
minSDK	30

2.2 Android architecture

To program a platform-specific application, it can be important to understand the basic architecture of the platform, especially when dealing with taks regarding memory or Bluetooth. This section will give a brief insight into the structure of the Android construction. Tasks discussed later in this thesis may reference this chapter to convey a better understanding.

[1] https://developer.android.com/studio/intro
[2] https://developer.android.com/studio/releases
[3] https://stackoverflow.com/questions/78125635/processing-classes-for-emulated-method-breakpoints-this-popup-comes-and-loadin
[4] https://developer.android.com/develop/connectivity/bluetooth/bt-permissions

The core of the Android Platform is a Linux based kernel. On top of Android's Kernel are four more stack layers, such as the Hardware Abstraction Layer, Native Libraries/Android Runtime, Java API Framework and the uppermost layer, the System App layer. (also see: Figure 1). Going into detail for each stack layer is not necessary for this work, consequently only essential layers like the Java API Framework will be considered in more depth. A short introduction into each layer is given by table 2 and figure 1 .

Table 2: Layers explanation

Layer	**Functionality**
System App's (Application Layer)	This layer is a collection of preinstalled applications. Apps contained in this collection, have the same status at system level as third-party apps(system settings is the only exception). Giving the user freedom for design.
Java API Framework (Framework Layer)	Enables acces to all features of the Android OS, such as resource manager and view system.
Native Libraries	System components such as ART and HAL are written in C/C++, those functionalities can be used via API's
Android Runtime (ART)	Every running application is an instance of ART. ART operates on the same layer as Native Libraries
Hardware Abstraction Layer (HAL)	Provides interfaces to the Java API Framework, hence addresing hardware components like Camera or Bluetooth module is possible.
Linux Kernel	Foundation of the OS. Administrates system resources, and communicates with hardware.

A layered archtitecture like the Android OS, ensures modularity, maintainability and security. Each layer is designed with a specific responsibility, and communicates primarly with adjacent layers. Applications interact with the Framework layer via the Android SDK (bridge between Application Layer and Framework Layer), which provides access to core system services. The Framework, in turn, relies on the Native Libraries and Runtime layers for functionalities like graphics rendering, media playback, and app execution. Hardware communication is abstracted through the Hardware Abstraction Layer (HAL), which interfaces with the Linux Kernel. This structured separation of concerns enhances the system's flexibility, allowing independent evolution and optimization of each layer.

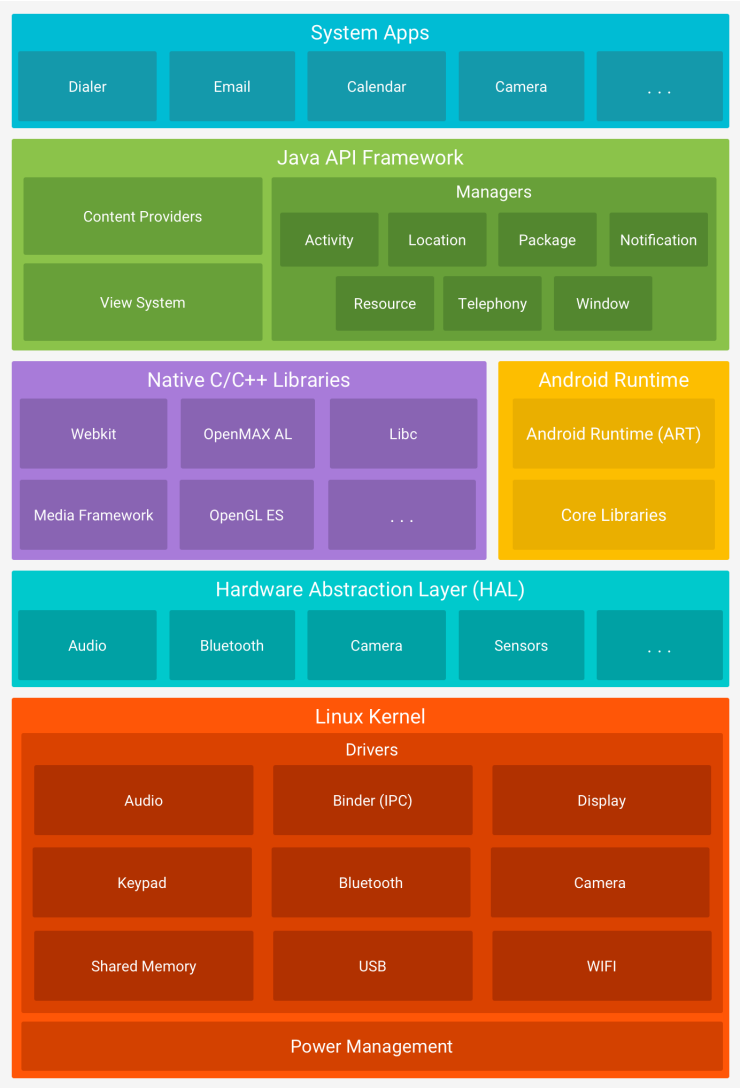


Figure 1: Android Platform architecture

2.2.1 Java Framework API (Framework Layer)

As already mentioned in 2.2, the Android Java Framework API is a collection of libraries and tools provided by Android to simplify the development of mobile applications. Built on top of Java, it enables developers to create apps with pre-built components, utilities, and services that interact with the Android operating system. The framework abstracts the complexity of low-level operations and provides high-level APIs for features such as UI design, data storage, networking, and hardware access.

To provide an impression, Figure 2 shows a code excerpt in which the Java API was implicitly used by androidx. To keep this chapter manageable, it can be simplified said that androidx is a superset of the Java API Framework.

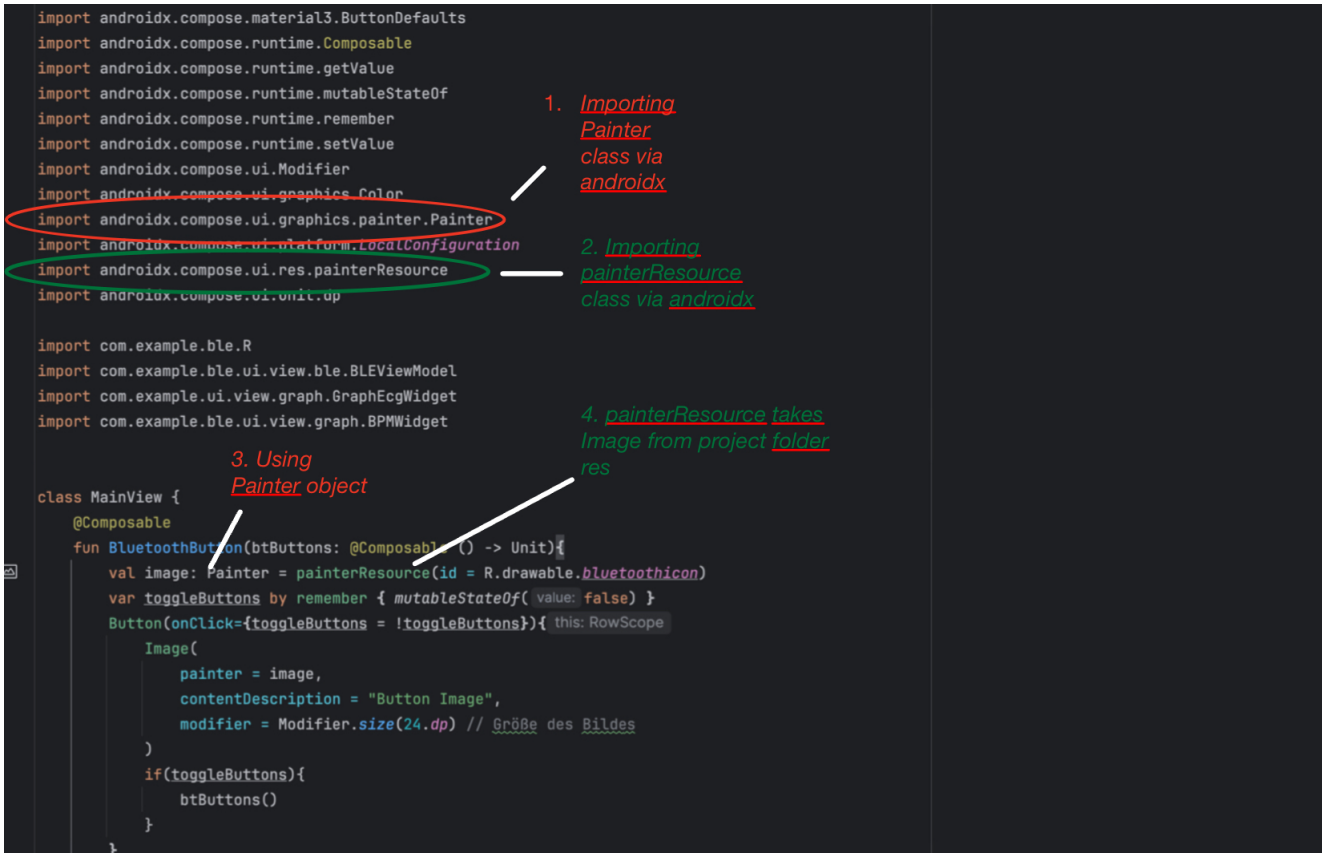


Figure 2: Android Java API example with androidx

Figure 2 provids a glimpse of the capabilities of the androidx library. Showing how to draw an image with the Painter class and utilizing the resource manager mentioned in table 2.

To beginn its elementary to import the Painter class aswell as the painterResource class. As shown in 1. and 2. in Figure 2. Then at 3. a Painter object is created, at 4. the Painter object is definied by the painterResource function. The painterResource function gets an id as an argument, to query the file from the res/drawable folder. The res folder contains all additional app resources (files, images, etc.). Finally we can draw the Image by passing the object from 3. to an Image function.

3 Functional Requirements

Requirement engineering is essential at the beginning of every project. Formulating and documenting precisely defined requirements is a cornerstone in the course of realization. In this way, mundane mistakes can be avoided and the vision of the final product is clear. Thus this chapter will explain all functional requirements of the finshed product.

3.1 Drawing ECG-Data in realtime

To realize a realtime display of someones hearthrate, it is helpful to split this rather complex functionality into multiple concerns:

- The sample rate of the microcontroller (1)
- Consume and process incoming data at once (2)
- Datahandling and Drawing must happen in parallel (3)

After gaining a concicse overview over the issues, its now import to view and solve every issue isolated and subsequently combine each solved problem with the other. It is important to mention that some issues may create restrictions for other issues. Therefore, it is essential to implement one task at a time and connect them all afterward.

We now look at our problems in a rather theoretically way, to facilitate the implementation in the end.

(1) Handle The Sample rate of the microcontroller

The microcontroller sends 10 datapieces every 8 milliseconds (1000ms = 1s).

$$\frac{1000}{8} = \frac{125}{1} \mathrel{\widehat{=}} 125Hz$$

Hence we have to process a sample rate of: $f(t) = 125 \times t$, t in seconds. If handling this datastream is not possible, data will be lost.

(2)Consume and process incoming data at the same time

To consume and process data at the same time, we split up processing and consuming into two indepent tasks by assigning each task to its own thread, making it possible to operate in paralell.

Since both threads have to access the same ressource at the same time, we have to make use of mutual exclusion. The concept of mutual exclusion will be explained in the later course of this thesis

(3)Datahandling and drawing in parallel

Thus the drawing entity has to countiously read its data, and the datahandler entity receive a continous data stream, one has to think of a fitting datastructure. The data structure must be able to constantly record new data and retain a certain amount of old data. So that a screen-width drawing can be displayed continuously. In addition, it is important that the data structure is not too large, which would lead to visible latencies.

This led to the Ringbuffer beeing the datastructure of choice. Ringbuffers are often used when having to process datastreams. Thus the old data gets overwritten by new data, the ringbuffer has a fixed size. Therefore, there is no need to worry about the data structure consuming too much memory.

3.2 24-Hour recording

The functional requirements of the 24-Hour recording are shown in the list below. All of the listed requirements will be explainend in detail at the Implementation chapter.

- Recording mode
- Filechooser conzept, to choose multiple files
- Start a recording, and assigning a name
- Abort a recording
- Display 24-Hour long recorded ECG-Data
- Show the current time intervalls
- Zoom into the displayed data, zomming out of the displayed data
- Plot BPM of the current diplayed time intervall

3.3 Connection via Bluetooth Low Energy

The application is required to establish and maintain a reliable Bluetooth Low Energy (BLE) connection with the hardware. A stable connection is essential for continous data-transfer aswell as the 24-Hour recording Mode. The BLE entity must be able of the following items:

- Continous datastream
- Communicate with the application

- Sharing current the current mode (realtime, or recording)
- Scan for near devices
- reassamble packages that are to large

4 Implementation

The first part of this chapter provides an overview of the connection between each entity, and the tasks of the given entitys. Aswell as an close look into the implementation of the functional requirements, by analysing code snippets of the final product. While the second part will explain every necessity to assemble all parts together for a working product.

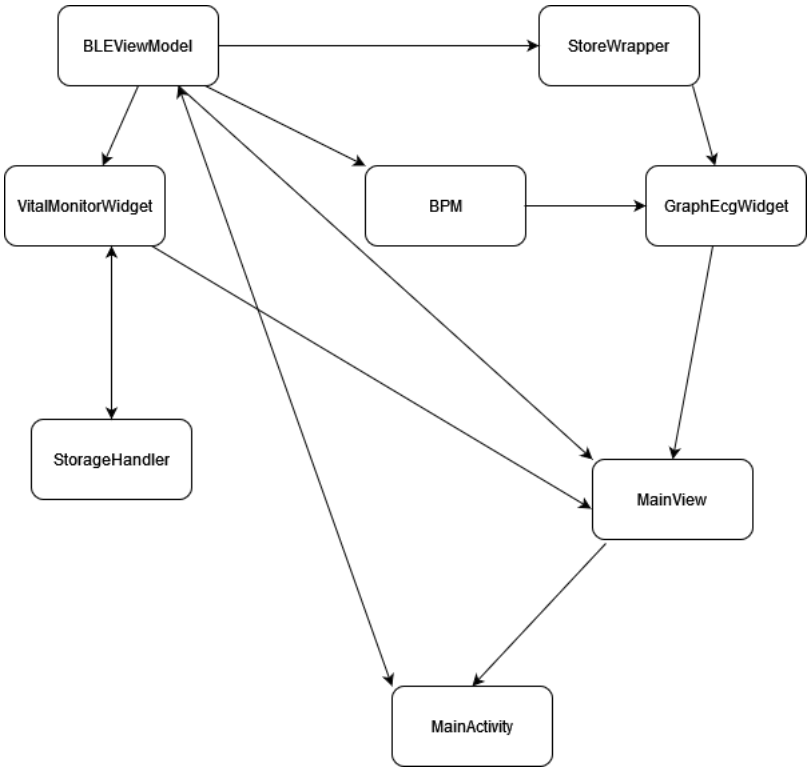
4.1 Definiton of Entitys

Table 3: Entiys area of responsibility

Layer	**Functionality**
BLEViewModel	• Communication with the microcontroller • Receives datastream • Sends instructions to microcontroller
GraphEcgWidget	Draws the ECG data in realtime, and displays Beats per minute
Storewrapper	Converts data from microcontroller into drawble data
BPM	Calculates the current Beats Per Minute
VitalMonitorWidet	Plots up to 24 Hour long recoreded ECG data and the BPM of the plotted Intervall. Provides zooming functionality.
StorageHandler	Allows usage of the OS filechooser, to select files.
MainView	Collection of visble elements (Widgets), to provide reusability
MainActivity	Visible to user, contains the complete User Interface. Takes and processes user inputs.

4.2 Inter-Entity Communication Overview

The following diagramm shows the Information flow between all entitys. The unidirectional arrow means that, information is flowing oneside. An informationexchange between two entitys is represented by an bidirectional arrow.



Literature references

S.J. Clarke. Selecting the correct tools and methods for software systems development. In *IEE Colloquium on Are Software Development Technologies Delivering Their Promise?*, page 7/1, 1995. doi: 10.1049/ic:19950387.

Andy Field. *Discovering Statistics Using SPSS*. SAGE Publications Ltd, 3 edition. ISBN 978-84787-906-6.

Mandahadi Kusuma, Ahmad Haris Rifani, and Bambang Sugiantoro. Comparison analysis of jetpack compose and flutter in android-based application development using technical domain. In *2023 Eighth International Conference on Informatics and Computing (ICIC)*, pages 3–5, 2023. doi: 10.1109/ICIC60109.2023.10381987.